

Mantenimiento de Software, Mejoras al Programa Sudoku

Alejandro Román Salazar

Tecnológico de Costa Rica

Taller de Programación

William Mata Rodríguez

29 de junio de 2026

Contenido

1. Enunciado del Proyecto	3
1.1 Objetivos.....	3
1.2 Resumen de Requerimientos	3
2. Temas Investigados.....	4
2.1 Software de Control de Versiones y Colaboración.....	4
2.2 Git y GitHub	5
2.3 Mejores Prácticas con Control de Versiones	5
2.4 Uso de Herramientas de IA	6
3. Solución: Detalle de las Mejoras Implementadas.....	9
3.1 Esquema de Cuentas de Usuario	9
3.2 Generación Automática de Partidas	10
3.3 Panel de Elementos Personalizado	10
3.4 Juegos Multinivel.....	11
3.5 Cambios al Top X: POO, ABB y Pickle	12
3.6 Funcionalidad Propia: Contador de Jugadas.....	13
4. Estructura de los Archivos Usados.....	14
usuarios.json (nuevo en este proyecto).....	14
sudoku2026_bitacora_jugadas.pkl (reemplaza el JSON anterior)	14
sudoku2026configuracion.json (modificado).....	14
sudoku2026partidas.json (eliminado)	14
sudoku2026juegoactual.json (sin cambios estructurales)	15
5. Conclusiones del Trabajo.....	16
5.1 Problemas Encontrados y Soluciones.....	16
5.2 Aprendizajes Obtenidos	17
6. Estadística de Tiempos	19
7. Lista de Revisión del Proyecto.....	20
Funcionalidades desarrolladas adicionalmente	21

1. Enunciado del Proyecto

1.1 Objetivos

Este proyecto consiste en una actividad de mantenimiento de software sobre el programa Sudoku desarrollado anteriormente. Los objetivos son:

- Realizar una actividad de mantenimiento de software de tipo mejoras: agregar nuevas funcionalidades y realizar cambios técnicos sobre un programa ya existente.
- Implementar el Árbol Binario de Búsqueda (ABB) como Tipo de Dato Abstracto, utilizando recursión en sus operaciones de inserción y recorrido.
- Aplicar conceptos de Programación Orientada a Objetos (POO): clases, atributos, métodos y objetos, mediante la creación de las clases Partida, NodoABB y ABB.
- Usar software de control de versiones y trabajo colaborativo (GitHub), llevando un historial de commits desde el inicio del desarrollo.
- Fomentar la habilidad de investigación sobre control de versiones, Git, GitHub y buenas prácticas de ingeniería de software relacionadas.

1.2 Resumen de Requerimientos

El programa de mantenimiento conserva toda la funcionalidad del Sudoku original (tablero interactivo, validaciones, pilas de deshacer/rehacer, cronómetro, guardar/cargar partida) y agrega seis mejoras nuevas:

- Esquema de cuentas: el acceso al programa ahora requiere una cuenta identificada por correo electrónico. El sistema genera un código de ingreso de 6 dígitos, lo encripta con SHA-256 y lo guarda en usuarios.json. Ya no se solicita el nombre del jugador manualmente en la opción Jugar.
- Generación automática de partidas: se eliminó el archivo de partidas pregrabadas. Ahora cada partida se genera en tiempo real mediante un algoritmo de backtracking recursivo, según el nivel de dificultad configurado.
- Panel de elementos ampliado: a las opciones de números y letras se agregó una tercera opción donde el jugador define sus propios 9 símbolos (caracteres únicos y diferentes entre sí).
- Juegos multinivel: una nueva opción de configuración permite que el jugador avance automáticamente de Fácil a Intermedio a Difícil al completar cada nivel, de forma cíclica.
- Cambios en el manejo del Top X: las partidas completadas se modelan como objetos de la clase Partida, almacenados en un Árbol Binario de Búsqueda (uno independiente por nivel) usando

inserción y recorrido recursivos. Los tres árboles se persisten en un archivo binario mediante el módulo pickle, reemplazando el JSON anterior.

- Funcionalidad propia: se agregó un contador de jugadas totales, visible en tiempo real durante la partida, que incrementa tanto con jugadas colocadas como con acciones de deshacer y rehacer.

2. Temas Investigados

2.1 Software de Control de Versiones y Colaboración

Un sistema de control de versiones (VCS, Version Control System) es una herramienta que registra los cambios realizados sobre un conjunto de archivos a lo largo del tiempo, permitiendo recuperar versiones anteriores, comparar diferencias entre versiones y, en el caso de los sistemas distribuidos, sincronizar el trabajo de varias personas sobre el mismo proyecto.

Su importancia en el desarrollo de software radica en varios aspectos: permite mantener un historial completo y reversible de la evolución del código, lo cual facilita identificar en qué momento se introdujo un error; habilita el trabajo colaborativo sin sobrescribir el trabajo de otros mediante mecanismos de fusión (merge); sirve como respaldo (backup) distribuido del proyecto; y documenta de forma natural la evolución del software a través de los mensajes de cada cambio registrado (commit).

Tres paquetes de software de control de versiones ampliamente utilizados son:

- Git: sistema de control de versiones distribuido, de código abierto, creado por Linus Torvalds en 2005 para el desarrollo del kernel de Linux. Es el más utilizado actualmente en la industria por su velocidad, su modelo de ramas (branches) ligero y su funcionamiento totalmente local (cada copia del repositorio contiene el historial completo).
- Subversion (SVN): sistema de control de versiones centralizado, más antiguo que Git. A diferencia de Git, depende de un único servidor central donde se almacena el historial; los desarrolladores deben estar conectados a ese servidor para registrar cambios. Aunque ha perdido popularidad frente a Git, todavía se usa en algunas empresas con infraestructura legada.
- Mercurial: sistema de control de versiones distribuido similar a Git en filosofía, también surgido en 2005. Se destaca por una interfaz de comandos considerada más simple y consistente que la de Git, aunque su adopción en la industria es considerablemente menor.

2.2 Git y GitHub

Git es el programa de control de versiones en sí mismo: una herramienta de línea de comandos que se instala localmente en la computadora y permite registrar (commit), comparar y revertir cambios en los archivos de un proyecto. Git funciona completamente sin conexión a internet, ya que cada copia del repositorio contiene el historial completo del proyecto.

GitHub, en cambio, es un servicio web (una plataforma en la nube) que aloja repositorios de Git de forma remota. GitHub no es lo mismo que Git: es una capa adicional que ofrece almacenamiento remoto, una interfaz visual para explorar el historial, herramientas de colaboración (issues, pull requests, revisión de código) y mecanismos de respaldo en la nube.

La diferencia esencial entre ambos puede resumirse así: Git es la herramienta que registra el historial de cambios en la máquina local; GitHub es el servicio donde ese historial se puede subir (push), compartir y respaldar de forma remota. Es posible usar Git sin GitHub (por ejemplo, con un repositorio puramente local), pero GitHub depende de Git como su motor de control de versiones subyacente.

En este proyecto se utilizó Git Bash (la terminal incluida en la instalación de Git para Windows) para ejecutar los comandos `git init`, `git add`, `git commit` y `git push`, conectando el repositorio local con un repositorio remoto creado en GitHub bajo el nombre `sudokuV2`.

2.3 Mejores Prácticas con Control de Versiones

Algunas buenas prácticas de ingeniería de software relacionadas con el uso de control de versiones, aplicadas en este proyecto, son:

- Iniciar el repositorio desde el principio del desarrollo: el spec exige usar GitHub desde el inicio del proyecto, no solo al final. Esto permite que el historial de commits refleje el progreso real, en vez de ser un único commit gigante hecho a último momento, lo cual no demuestra evolución incremental.
- Mensajes de commit descriptivos: cada commit debe llevar un mensaje breve pero claro que explique qué cambió, por ejemplo 'Implementado modo multinivel y corregidos bugs de validacion' en vez de mensajes genéricos como 'cambios' o 'arreglos'. Esto facilita entender el historial sin tener que leer el código línea por línea.
- Commits frecuentes y atómicos: es preferible hacer varios commits pequeños, cada uno representando un cambio coherente (por ejemplo, una mejora terminada), en vez de un único commit con muchos cambios mezclados. Esto facilita identificar en qué commit se introdujo un error si algo falla más adelante.

- Configurar la identidad del autor: usar `git config --global user.name` y `user.email` para que cada commit quede correctamente atribuido a quien lo realizó, lo cual es esencial en proyectos colaborativos.

2.4 Uso de Herramientas de IA

Durante el desarrollo de este proyecto se utilizó Claude (Anthropic) como herramienta de apoyo tutorial, siguiendo el mismo enfoque consciente, crítico y ético del proyecto anterior: explicación de conceptos y guía paso a paso, con el código escrito por el propio estudiante.

Objetivo del uso	Entender cómo implementar un Árbol Binario de Búsqueda (ABB) con recursión, conectándolo con los conceptos de POO ya vistos en clase
Herramienta utilizada	Claude (Anthropic) — claude.ai
Prompt o pregunta	¿Cómo funciona la inserción recursiva en un Árbol Binario de Búsqueda? ¿Por qué falla si llamo a un método con guión bajo sin el guión bajo, o si lo defino dentro de otro método?
Respuesta	Explicación de la lógica de comparación (menor va a la izquierda, mayor o igual a la derecha) y de la recursión necesaria para descender por el árbol hasta encontrar una posición vacía. También explicación de scope: un método anidado dentro de otro método de una clase no es accesible como <code>self.metodo()</code> desde el resto de la clase.
¿Cómo usó o adaptó la respuesta?	Usé la explicación para escribir yo mismo la clase ABB completa (<code>insertar_nodo</code> , <code>insertar_recursivo</code> , <code>recorrer_arbol</code> , <code>recorrer_recursivo</code>). Cuando tuve un error de <code>AttributeError</code> porque había anidado mal un método, la explicación de scope me permitió identificar y corregir el problema sin que se me diera la solución directamente.
Reflexión crítica	Fue clave para entender la diferencia entre un método de instancia normal y una función anidada dentro de otro método, error que cometí dos veces durante el proyecto y que después pude identificar solo viendo el <code>traceback</code> .

Objetivo del uso	Depurar un bug donde el tablero generado quedaba con números repetidos al jugar, aunque la lógica de validación parecía correcta
Herramienta utilizada	Claude (Anthropic) — claude.ai
Prompt o pregunta	Las funciones de validación de fila, columna y cuadrícula están bien escritas pero no detectan jugadas inválidas. ¿Cómo encuentro dónde está el problema real?
Respuesta	Sugerencia de usar print() y type() para inspeccionar en tiempo de ejecución qué tipo de dato estaba llegando como 'elemento' a la función de validación, en vez de asumir directamente dónde estaba el error.
¿Cómo usó o adaptó la respuesta?	Seguí la sugerencia agregando prints temporales en validar_fila(). Así descubrí que numero_seleccionado.get() devolvía un string ('2') incluso cuando el jugador seleccionaba un número, mientras el tablero guardaba enteros (2), por lo que la comparación 'elemento in tablero[fila]' nunca encontraba coincidencias. Yo mismo escribí la solución (convertir con int() cuando el string es numérico, usando isdigit()).
Reflexión crítica	Aprendí una técnica de depuración que pienso seguir usando: en vez de adivinar dónde está el bug, usar prints estratégicos para confirmar hipótesis sobre tipos de datos y valores en tiempo de ejecución antes de modificar código.

Objetivo del uso	Aprender los comandos básicos de Git para inicializar y conectar un repositorio local con GitHub por primera vez
Herramienta utilizada	Claude (Anthropic) — claude.ai
Prompt o pregunta	Nunca he usado Git ni GitHub. ¿Qué pasos sigo desde cero para crear un repositorio, conectarlo con GitHub y subir mi proyecto?
Respuesta	Secuencia de pasos: crear cuenta en GitHub, instalar Git, configurar identidad (git config --global), crear repositorio remoto, ejecutar git init, git add ., git commit -m, git remote add origin, y git push -u origin master para el primer push.
¿Cómo usó o adaptó la respuesta?	Seguí los pasos exactamente como guía para mi primera vez usando control de versiones, ya que era un proceso completamente nuevo para mí. Repetí git add, git commit y git push en cada avance significativo del proyecto, escribiendo yo mismo los mensajes descriptivos de cada commit.
Reflexión crítica	Aprendí el patrón básico de trabajo con Git que no conocía previamente: la diferencia entre el área de preparación (staging con add), el registro permanente (commit) y la sincronización remota (push). También entendí por qué Git Bash crea una ventana adicional vacía si no se oculta la ventana raíz de tkinter al combinarlo con mi programa.

2.5 Referencias

Parte del aprendizaje de este proyecto fue gracias al canal de YouTube “Código Espinoza - Automatiza tu Vida” dicho canal tiene una playlist de nombre “Curso de Tkinter: Crea Interfaces Gráficas en Python Desde Cero Hasta Avanzado”. Esta playlist la vi de a poco durante el proyecto anterior y durante este para poder así tener conocimiento de alguien que no es una IA. Aquí la referencia:

Código Espinoza - Automatiza tu Vida. (2023, 28 de mayo). Curso de Tkinter: Crea Interfaces Gráficas en Python Desde Cero Hasta Avanzado [Video]. YouTube.

<https://www.youtube.com/watch?v=6frbSDzvFVQ>

3. Solución: Detalle de las Mejoras Implementadas

3.1 Esquema de Cuentas de Usuario

Nombre de la mejora (según especificación): 'Nueva funcionalidad: uso de cuentas para acceder a la aplicación.'

Detalle de la mejora: se reemplazó el acceso libre al programa por un esquema de autenticación basado en correo electrónico y un código numérico de un solo uso. Al ejecutar el programa, antes de mostrar el menú principal, aparece una ventana de inicio de sesión que solicita el correo electrónico del jugador.

Cómo se implementó: se creó el archivo usuarios.json con una lista de diccionarios, cada uno representando una cuenta (id, correo, código_ingreso encriptado, nombre y fecha de creación). Si el correo ingresado no existe en el archivo, el programa pregunta si se desea crear una cuenta nueva; de aceptar, se solicita el nombre mediante un cuadro de diálogo (simpledialog.askstring) y se registra el usuario con un id autoincremental. En ambos casos (usuario nuevo o existente) se genera un código aleatorio de 6 dígitos con random.randint(), se encripta con hashlib.sha256() antes de guardarlo, y se muestra al usuario simulando su envío por correo (mediante un mensaje en pantalla, ya que el proyecto no cuenta con un servidor de correo real). El usuario debe ingresar ese código, que se vuelve a encriptar y se compara contra el hash guardado para validar el acceso.

Cambios realizados en el programa: se reorganizó por completo el punto de entrada del programa. La creación de la ventana principal se convirtió en la función abrir_menu_principal(nombre_jugador, raiz_oculta), que ahora se invoca únicamente después de un login exitoso, en vez de crearse directamente al iniciar el archivo. El nombre del jugador autenticado se guarda en una variable global (nombre_jugador_global) para que esté disponible en jugar(), guardar_partida_actual() y cargar_partida_actual() sin necesidad de pedirlo manualmente por un campo de texto, eliminando así esa solicitud redundante en la opción Jugar.

3.2 Generación Automática de Partidas

Nombre de la mejora: 'Generar partidas automáticamente.'

Detalle de la mejora: se eliminó por completo la dependencia del archivo sudoku2026partidas.json con partidas pregrabadas por el profesor. En su lugar, cada vez que el jugador presiona INICIAR JUEGO, el programa genera una partida nueva y distinta en tiempo real.

Cómo se implementó: se programó un algoritmo de backtracking recursivo. La función generar_tablero(elementos) crea una matriz 9x9 vacía y llama a llenar_tablero(tablero, elementos), que recorre las casillas y, para cada una vacía, prueba los 9 elementos disponibles (números, letras o símbolos personalizados) en orden aleatorio, validando cada intento con las funciones de validación de fila, columna y subcuadrícula ya existentes. Si un elemento es válido, se coloca y la función se llama a sí misma para continuar con la siguiente casilla; si en algún punto ningún elemento funciona, la función retorna False y la llamada anterior deshace su elección, retrocediendo (backtracking) para probar otra alternativa. Una vez generado el tablero completamente resuelto, la función quitar_casillas(tablero, nivel) elige posiciones al azar y las vacía hasta alcanzar la cantidad correspondiente al nivel de dificultad (35 para fácil, 45 para intermedio, 55 para difícil).

Cambios realizados en el programa: se eliminaron las referencias al archivo de partidas pregrabadas (la función cargar_partida() y su uso en jugar()). Dentro de iniciar_juego(), el bloque que antes seleccionaba una partida aleatoria del JSON fue reemplazado por la llamada a generar_tablero() y quitar_casillas().

3.3 Panel de Elementos Personalizado

Nombre de la mejora: 'Panel de elementos.'

Detalle de la mejora: se agregó una tercera opción de elementos en Configurar, además de números y letras: elementos definidos por el jugador. La restricción definida para estos elementos es que deben ser exactamente 9 caracteres de un solo símbolo cada uno, todos diferentes entre sí (por ejemplo: signos de puntuación o símbolos aritméticos como “?!@#\$\$%^&*”).

Cómo se implementó: en la pantalla de Configurar se agregó un campo de texto donde el jugador escribe sus 9 símbolos seguidos, sin espacios. La función `validar_elementos_personalizados(texto)` verifica dos condiciones: que el texto tenga exactamente 9 caracteres, y que al convertirlo a un conjunto (set), su tamaño siga siendo 9 (lo cual confirma que no hay caracteres repetidos, dado que un set elimina automáticamente los duplicados). Si la validación pasa, el texto se guarda en una nueva llave del archivo de configuración (`elementos_personalizados`) y, en `jugar()`, se convierte a una lista de caracteres individuales con `list(texto)` para alimentar tanto el panel de selección como el generador de partidas, reutilizando exactamente la misma lógica ya construida para números y letras.

Cambios realizados en el programa: se amplió el bloque condicional que determina los elementos del panel en `jugar()` con un tercer caso (`elif config['elementos'] == 'personalizado'`), se agregó la llave correspondiente a `CONFIG_DEFAULT`, y se modificó `guardar_config()` dentro de `configurar()` para validar y persistir el nuevo campo. Como parte de esta mejora se completó también el soporte de la opción letras, que había quedado pendiente del proyecto anterior, cambiando la variable `numero_seleccionado` de un `IntVar` a un `StringVar` para poder almacenar tanto números como letras o símbolos.

3.4 Juegos Multinivel

Nombre de la mejora: 'Juegos multinivel.'

Detalle de la mejora: se agregó la opción 'multinivel' al nivel de dificultad. Cuando está activa, el jugador empieza en Fácil; al completar exitosamente esa partida, el programa automáticamente prepara una partida nueva en Intermedio, lista para que el jugador presione INICIAR JUEGO. Al completar Intermedio, avanza a Difícil; al completar Difícil, regresa cíclicamente a Fácil, repitiéndose hasta que el jugador decida terminar.

Cómo se implementó: se agregó una variable `caso_multinivel` (`StringVar`) que recuerda en qué nivel real del ciclo se encuentra el jugador, independiente de la palabra 'multinivel' configurada (que identifica el modo, no una dificultad jugable). La función `avanzar_nivel(caso_multinivel)` usa el índice de la lista `NIVELES` y el operador módulo (`% 3`) para rotar cíclicamente entre los tres niveles reales: $(\text{índice_actual} + 1) \% 3$. Cuando el tablero se completa y el modo configurado es multinivel, se llama a esta función, se informa al jugador del nuevo nivel, se reinician visualmente el tablero (recreando los 81 botones vacíos) y el cronómetro (`segundos.set(0)` y la actualización del label correspondiente), y se desactiva la bandera de juego iniciado para que el jugador controle cuándo continuar.

Cambios realizados en el programa: se modificó la pantalla de Configurar para incluir 'multinivel' en la lista de niveles. Dentro de `colocar_numero()` e `iniciar_juego()`, se agregó la distinción entre el nivel configurado (`nivel`) y el nivel real jugado en cada momento (`nivel_real`), usado para generar la partida correcta y registrar la dificultad verdadera en la bitácora de tiempos.

Nota sobre alcance: el spec también solicita, para el caso de multinivel combinado con temporizador (cuenta regresiva), permitir configurar tiempos máximos independientes para cada uno de los tres niveles. Esta parte específica no fue implementada, dado que el temporizador de cuenta regresiva en general (ya identificado como pendiente desde el proyecto anterior) no llegó a desarrollarse por restricciones de tiempo. El cronómetro normal (de conteo ascendente) sí funciona correctamente en modo multinivel.

3.5 Cambios al Top X: POO, ABB y Pickle

Nombre de la mejora: 'Cambios para el manejo del Top X.'

Detalle de la mejora: se reemplazó por completo el almacenamiento de partidas completadas. En vez de un diccionario en un archivo JSON ordenado manualmente al generar el reporte, ahora cada partida es un objeto de la clase `Partida`, almacenado dentro de un Árbol Binario de Búsqueda (ABB), con un árbol independiente por cada nivel de dificultad (un 'bosque' de tres ABB: `abb_facil`, `abb_intermedio`, `abb_difícil`).

Cómo se implementó: se definieron tres clases siguiendo exactamente la estructura propuesta en el enunciado. La clase `Partida` almacena los atributos `jugador`, `nivel`, `tiempo` y `fecha_hora` en su constructor, y su método `get_partida()` retorna un string formateado (incluyendo la conversión del tiempo en segundos a formato horas:minutos:segundos). La clase `NodoABB` encapsula una referencia a un objeto `Partida` junto con sus referencias a los nodos hijo izquierdo y derecho. La clase `ABB` implementa `insertar_nodo()`, que crea el nodo raíz si el árbol está vacío o delega en el método recursivo `insertar_recursivo()`, el cual compara el tiempo de la nueva partida contra el nodo actual: si es menor, desciende (o inserta) por la izquierda; si es mayor o igual, por la derecha. El método `recorrer_arbol()` inicia un recorrido en-orden mediante `recorrer_recursivo()`, visitando primero el subárbol izquierdo, luego el nodo actual (agregando el resultado de su `get_partida()` a una lista), y finalmente el subárbol derecho — garantizando que las partidas queden ordenadas de menor a mayor tiempo sin necesidad de un ordenamiento posterior explícito.

Los tres árboles se serializan completos (como una tupla) mediante el módulo `pickle` en el archivo binario `sudoku2026_bitacora_jugadas.pkl`, reemplazando el antiguo archivo JSON de bitácora. Cuando se completa una partida, `guardar_en_bitacora()` crea el objeto `Partida` correspondiente y lo inserta en el árbol del nivel correcto; al generar el reporte TOP X, `generar_topx()` carga los tres árboles, recorre cada uno con

recorrer_arbol() (obteniendo directamente las partidas ya ordenadas) y construye el PDF con reportlab a partir de esa lista, respetando el límite configurado en top_x.

3.6 Funcionalidad Propia: Contador de Jugadas

Nombre de la mejora: 'Funcionalidad definida por el programador.'

Detalle de la mejora: se agregó un contador visible en tiempo real durante la partida que totaliza el número de acciones realizadas por el jugador sobre el tablero. A diferencia de un simple contador de números colocados, este contador suma también las acciones de deshacer y rehacer jugada, reflejando el esfuerzo total invertido en resolver la partida, no solamente el avance neto.

Cómo se implementó: se agregaron dos variables en jugar(): contador_jugadas (IntVar, el número interno) y texto_contador (StringVar, el texto visible con formato 'Jugadas: X'). Un Label conectado a texto_contador mediante textvariable se actualiza automáticamente sin necesidad de llamar a configure() manualmente. El incremento se replica en tres puntos del programa: al colocar un número válido en colocar_numero(), al deshacer una jugada en deshacer_jugada(), y al rehacer una jugada en rehacer_jugada(), en los tres casos siguiendo el mismo patrón de actualizar primero el IntVar y luego reconstruir el texto del StringVar.

4. Estructura de los Archivos Usados

usuarios.json (nuevo en este proyecto)

Lista de diccionarios, cada uno representando una cuenta de usuario.

Campo	Tipo	Descripción
id	int	Identificador único, autoincremental empezando en 1
correo	string	Correo electrónico, único por usuario
código_ingreso	string	Hash SHA-256 del código de 6 dígitos vigente
nombre	string	Nombre del jugador, único por usuario
fecha_creación	string	Fecha/hora de creación, formato ISO8601 básico

sudoku2026_bitacora_jugadas.pkl (reemplaza el JSON anterior)

Archivo binario generado con el módulo pickle. Contiene una tupla con tres objetos de la clase ABB (abb_facil, abb_intermedio, abb_dificil), cada uno almacenando internamente sus nodos (NodoABB) y las partidas (objetos Partida) ordenadas por tiempo mediante la estructura del árbol.

sudoku2026configuracion.json (modificado)

Se mantiene la estructura del proyecto anterior, agregando la llave 'elementos_personalizados' (string) que guarda los 9 símbolos definidos por el jugador cuando se elige esa opción, y se amplía la llave 'nivel' para aceptar también el valor 'multinivel'.

sudoku2026partidas.json (eliminado)

Este archivo, usado en el proyecto anterior para almacenar partidas pregrabadas, fue eliminado según lo solicitado en la especificación. Las partidas ahora se generan en tiempo real mediante el algoritmo de backtracking.

sudoku2026juegoactual.json (sin cambios estructurales)

Conserva la misma estructura del proyecto anterior: diccionario con el nombre del jugador como llave, almacenando el nivel, el tablero completo y la partida fija de la sesión guardada.

5. Conclusiones del Trabajo

5.1 Problemas Encontrados y Soluciones

Problema 1: Método de clase definido dentro de otro método

Al implementar la clase ABB, el método `insertar_recursivo()` fue escrito accidentalmente anidado dentro de `insertar_nodo()`, en vez de como un método independiente al mismo nivel dentro de la clase. Esto generó un `AttributeError: 'ABB' object has no attribute 'insertar_recursivo'` al intentar llamarlo con `self.insertar_recursivo(...)`, ya que un método definido dentro de otro método no queda registrado como atributo de la clase.

Solución: se identificó el problema revisando cuidadosamente la indentación de ambos métodos, y se corrigió moviendo `insertar_recursivo()` para que quedara al mismo nivel de indentación que `insertar_nodo()`, como un método hermano dentro de la clase.

Problema 2: Comparación entre tipos de datos incompatibles (string vs entero)

Al ampliar el soporte de elementos para incluir letras y símbolos personalizados, fue necesario cambiar la variable `numero_seleccionado` de un `IntVar` a un `StringVar` (ya que un `IntVar` solo admite números enteros). Esto introdujo un bug sutil: el tablero seguía almacenando enteros para las casillas numéricas, pero `numero_seleccionado.get()` ahora devolvía siempre un string, incluso para los números. La comparación `elemento in tablero[filas]` dentro de las validaciones nunca encontraba coincidencias, porque comparaba un string ('2') contra una lista de enteros (2), permitiendo jugadas inválidas sin marcar error.

Solución: se agregó una conversión condicional en `colocar_numero()`: si el valor seleccionado consiste únicamente en dígitos (verificado con el método `.isdigit()` de los strings), se convierte a entero con `int()` antes de pasar por las validaciones; si no (por ejemplo, una letra o símbolo), se mantiene como string.

Problema 3: Múltiples ventanas raíz de tkinter al combinar login con el menú principal

Al introducir la ventana de inicio de sesión (creada con `tk.Toplevel()` antes de que existiera ninguna ventana `tk.Tk()`), tkinter generaba automáticamente una ventana raíz invisible adicional que terminaba siendo visible en pantalla, mostrando una ventana vacía junto a la de login.

Solución: se creó explícitamente esa ventana raíz al inicio de `iniciar_sesion()` (`raiz_oculta = tk.Tk()`) y se ocultó inmediatamente con `.withdraw()`. Esa misma ventana se reutiliza después como la ventana principal del programa: en `abrir_menu_principal()`, en vez de crear una nueva con `tk.Tk()`, se reutiliza la raíz oculta y se vuelve a mostrar con `.deiconify()`.

Problema 4: Crecimiento descontrolado en la cantidad de parámetros de las funciones

A medida que se agregaron las mejoras (multinivel, contador de jugadas, elementos personalizados), funciones como `iniciar_juego()` y `colocar_numero()` acumularon una cantidad considerable de parámetros (más de 15 en algunos casos), volviéndose difíciles de leer y propensas a errores de orden o de parámetros faltantes al momento de modificarlas.

Solución aplicada: dado el tiempo disponible, se optó por continuar agregando parámetros de forma explícita en vez de refactorizar la arquitectura completa, priorizando la entrega funcional sobre el diseño ideal. Como aprendizaje reconocido, la solución correcta a largo plazo sería encapsular el estado completo de una partida (tablero, pilas, banderas, variables de tkinter) en una clase propia, similar al enfoque de POO ya aplicado en el Top X con la clase ABB, y pasar un único objeto de esa clase entre funciones en vez de decenas de parámetros sueltos.

5.2 Aprendizajes Obtenidos

- Árboles Binarios de Búsqueda y recursión aplicada: implementar el ABB permitió entender en la práctica por qué la recursión es natural para estructuras jerárquicas como los árboles — cada subárbol se resuelve con la misma lógica que el árbol completo, descendiendo hasta encontrar el caso base (nodo nulo).
- Programación Orientada a Objetos como organización del estado: las clases Partida, NodoABB y ABB mostraron cómo encapsular datos y comportamiento relacionado en una sola unidad, en contraste con el resto del programa que sigue un enfoque más procedural con muchas funciones y variables sueltas — diferencia que quedó evidenciada en el Problema 4 de esta misma sección.
- Persistencia de objetos con pickle: a diferencia de JSON, que solo serializa estructuras de datos simples, pickle permite guardar y recuperar objetos completos de clases personalizadas (como los tres ABB), preservando su estructura interna sin necesidad de reconstruirla manualmente.

- Control de versiones con Git y GitHub: primera experiencia trabajando con un sistema de control de versiones real, incluyendo la configuración de identidad, la secuencia básica de comandos (init, add, commit, push) y la importancia de hacer commits frecuentes con mensajes descriptivos en vez de un solo commit final.
- Depuración basada en evidencia más que en intuición: la técnica de usar print() y type() para confirmar hipótesis sobre el comportamiento real del programa (en vez de adivinar la causa de un bug) se consolidó como un hábito de trabajo durante este proyecto, resolviendo errores que de otra forma habrían sido difíciles de localizar.
- Costo del mantenimiento de software: experimentar en carne propia cómo agregar funcionalidades a un programa ya existente generó efectos en cadena sobre código que ya funcionaba (cambios de tipo de dato, parámetros nuevos propagándose por múltiples funciones) ilustró de forma concreta por qué el mantenimiento de software es una actividad compleja y por qué el buen diseño inicial reduce ese costo a futuro.

6. Estadística de Tiempos

Actividad Realizada	Horas
Análisis del problema y lectura del spec	2
Diseño de las clases Partida, NodoABB y ABB	2
Investigación de Git, GitHub y control de versiones	2
Investigación de POO, recursión en árboles y pickle	2
Programación — Clases POO y persistencia con pickle (Top X)	3
Programación — Esquema de cuentas (login, hashlib, usuarios.json)	3
Programación — Generación automática de partidas (backtracking)	2
Programación — Panel de elementos personalizado	1.5
Programación — Juegos multinivel	2
Programación — Funcionalidad propia (contador de jugadas)	1
Configuración de Git/GitHub y commits	1
Pruebas y corrección de errores (bugs de tipos, scope, indentación)	5
Documentación interna del programa	1.5
Elaboración del manual de usuario actualizado	2
Elaboración de documentación del proyecto	3
TOTAL	33

7. Lista de Revisión del Proyecto

Concepto	Puntos	Puntos obtenidos	% Avance	Análisis de resultados
Esquema de cuentas para acceder a la aplicación	20	20	100%	
Generar partidas automáticamente	10	10	100%	
Panel de elementos	10	10	100%	Incluye números, letras y elementos definidos por el jugador, todos validados.
Juegos multinivel	10	9	90%	El avance cíclico entre niveles funciona completo, incluyendo reinicio de tablero y cronómetro. No se implementó la configuración de tiempos máximos independientes por nivel para el modo timer, ya que el temporizador de cuenta regresiva en general no fue desarrollado (pendiente desde el proyecto anterior).
Cambios para el manejo del Top X	40	40	100%	Clases Partida, NodoABB y ABB con inserción y recorrido recursivos; tres árboles independientes por nivel; persistencia con pickle; reporte en PDF generado a partir del recorrido en-orden.
Funcionalidad definida por el programador	10	10	100%	Contador de jugadas totales en tiempo real, incluyendo deshacer y rehacer.
TOTAL	100	99		

Funcionalidades desarrolladas adicionalmente

- Se completó el soporte de la opción 'letras' en el panel de elementos, que había quedado pendiente sin funcionar correctamente desde el proyecto anterior.
- Se corrigieron varios bugs heredados del proyecto anterior detectados durante la revisión oral y durante las pruebas de esta entrega: validación incorrecta de casillas fijas, error en la pila de deshacer/rehacer al no recibir la referencia de los botones, y el cronómetro que no se detenía al completar una partida.
- Se reorganizó el flujo de inicio del programa para integrar correctamente el login con la ventana principal, evitando la aparición de ventanas duplicadas de tkinter.